Automatic Classification of Martian Surface Features Using Convolutional Neural Networks

Pradnya Chandrakant Bhakare
Course: Deep Learning
Queen Mary University of London

***Abstract*—This project proposes a convolutional neural network (CNN) for the automatic classification of Martian surface features based on HiRISE orbital images. The proposed CNN classifier categorizes Martian surface images into eight features: other, crater, dark dune, slope streak, bright dune, impact ejecta, swiss cheese, and spider features. The pro- posed CNN classifier shows a test accuracy of 90.85%. The proposed CNN classifier shows that deep learning can effectively aid the analysis of planetary surface features.**

I. Introduction

Significant amounts of high-resolution images are being generated from planetary exploration missions, and the images need to be processed efficiently to detect geological features. The HiRISE camera on the Mars orbiting spacecraft can detect high-resolution features on the Martian surface, which may include craters, dunes, and seasonal features that can give valuable information regarding geological activities on the Martian surface. It is very hard to manually inspect images.

Recent developments in deep learning, especially CNN architectures, have the potential to automatically detect spatial features directly from images. CNN models have shown good results in image recognition tasks, particularly because CNN models have the ability to effectively learn the hierarchical representation of the image. In this study, the effectiveness of CNN models was considered for the automatic classification of Martian surface features. Specifically, the study considers the development of an efficient CNN model, the application of appropriate preprocessing techniques, and the evaluation of the CNN model.[2]

II. Dataset and Preprocessing

The dataset contains more than 73,000 high–resolution Mars surface images labelled into eight geological categories. Class labels were converted into numerical indices to enable supervised learning using a convolutional neural network.[1]

Each image belongs to one of the following classes:

- Other
- Crater
- Dark Dune
- Slope Streak
- Bright Dune
- Impact Ejecta
- Swiss Cheese
- Spider

All the images were resized to 128×128 pixels and normalized in the range $[0, 1]$. An 80/20 split was used for stratified splitting so that the proportion of terrain categories was maintained in the training and test sets. Data augmentation was also used for generalization.[6]

A. Input Examples

Figure 1 shows examples of original images and their corresponding preprocessed inputs used by the CNN. This is because a comparison between the original and the preprocessed images needed to be carried out in order to ensure that the images were properly processed without losing important features. The images were resized to 128×128 pixels, normalized to the range $[0, 1]$, and augmented where necessary, all in a bid to enhance stability in the training process and ensure that the model did not concentrate on irrelevant surface features.

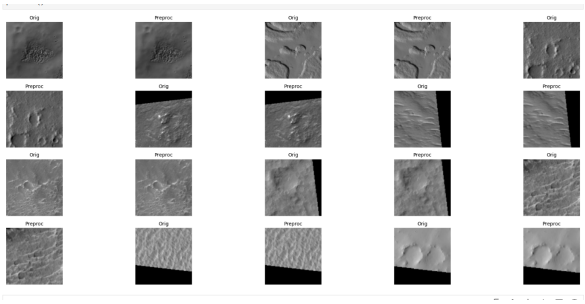

Fig. 1: Original and preprocessed Mars surface images.

B. Class Distribution

The class distribution plot in **Figure 2** displays the number of images available in the dataset for each of the Martian surface features. It can be noted from the plot that the number of images for classes *Other* and *Crater* is the highest, while for other classes like Slope Streak, Swiss Cheese, Dark Dune, and Impact Ejecta, the number of images is relatively lower. However, to make sure that the training and test sets have an equal number of images from all classes, stratified sampling was performed during the 80/20 split, which in turn helps the CNN learn effectively for all the classes of Martian terrain.

The class imbalance was attempted to be solved using class weights during training, where more importance was given to less represented classes like Slope Streak, Swiss Cheese, Dark Dune, and Impact Ejecta. While using class weights improved the model's focus on less represented classes, it also led to a form of overfitting in the model, as seen from a widening gap in training and validation accuracy and fluctuations in loss values. Hence, class imbalance is a problem that needs to be solved but in a manner that does not cause overfitting in the model. Figure 2 shows the number of images per class in the dataset.

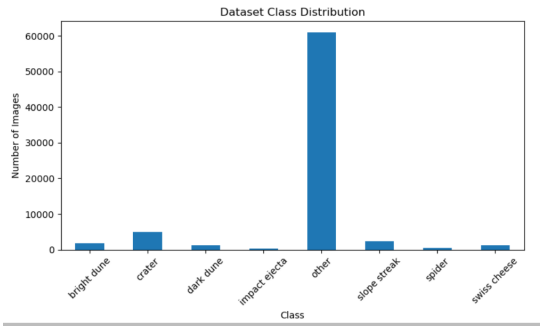

Fig. 2: Number of images in each geological category.

C. Network Design

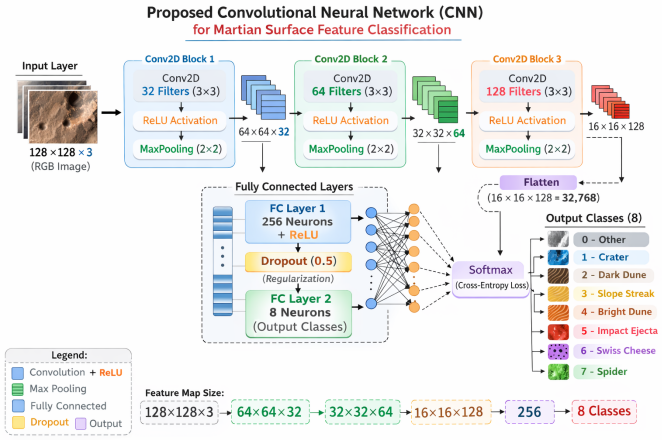

Fig. 3: Convolutional neural network architecture used for terrain classification.

The suggested convolutional neural network architecture will progressively extract spatial features from Martian images through three convolutional blocks followed by fully connected classification layers.[2]

Input Layer: The input images are represented as 128x128x3 tensors. Although the original images are in grayscale, the RGB representation has been preserved for convenience in implementing standard convolutional neural networks.

Convolutional Layers: There are three convolutional layers, each with 32, 64, and 128 filters, respectively, to learn hierarchical representations of the input data. Early convolutional layers are likely to learn basic features, including edges and textures, while deeper convolutional layers are likely to learn more abstract representations, including complex geological features such as craters and dunes. A 3x3 kernel has been selected to balance feature extraction and computational efficiency.

Activation Function: A ReLU activation function has been selected to introduce non-linearity in the model, thereby enabling efficient optimization.

Max Pooling: A 2x2 max pooling layer has been selected to reduce the spatial resolution of the image, thereby enabling efficient feature extraction.

Fully Connected Layers: Flattened feature maps are passed to a dense layer of 256 neurons to combine learned representations for classification.

Dropout Regularization: A dropout rate of 0.5 reduces overfitting by preventing co-adaptation of neurons during training.

Output Layer: The final softmax layer contains eight neurons corresponding to the terrain categories and is trained using categorical cross-entropy loss.

III. Training Procedure

The dataset was divided into 80% training and 20% testing samples using stratified sampling. Training settings were:

- Loss Function: Cross-Entropy Loss
- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 32
- Epochs: 17

IV. Results

A. Accuracy vs Epochs

Fig. 4: Training and testing accuracy across epochs.

Training accuracy increased steadily and converged near 91%. Validation accuracy closely followed the training curve, indicating good generalisation.[8][9]

B. Loss vs Epochs

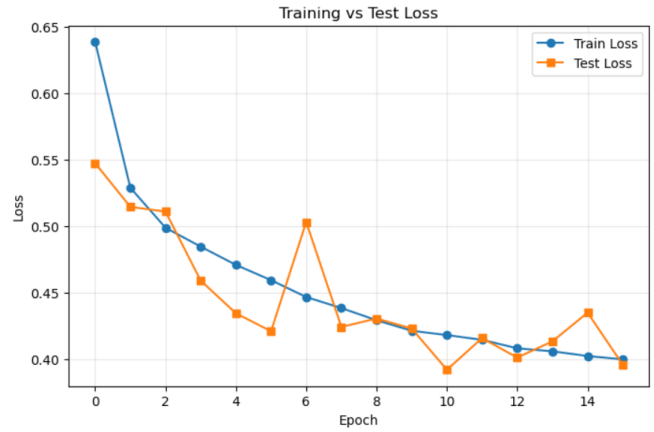

Fig. 5: Training and testing loss across epochs.

Overfitting was evaluated by comparing training and test loss. Both curves decreased consistently and remained closely aligned, showing stable learning behaviour.[3]

Final Model Performance:

- Train Accuracy: 0.9079
- Test Accuracy: 0.9085
- Train Loss: 0.3999
- Test Loss: 0.3958
- Loss Difference (Test – Train): -0.0041

The small loss difference between training and validation sets confirms that the model generalizes effectively. It shows minimal signs of overfitting, demonstrating stable performance across unseen data.[4][9]

C. Test Predictions

Figure 6 shows a set of 20 test images and their corresponding true labels and predictions. It is seen that the CNN model correctly classifies 18 test images and makes wrong predictions for only 2 test images. This shows the excellent ability of the model in recognizing unseen test images despite their possible similarity in terrain features.

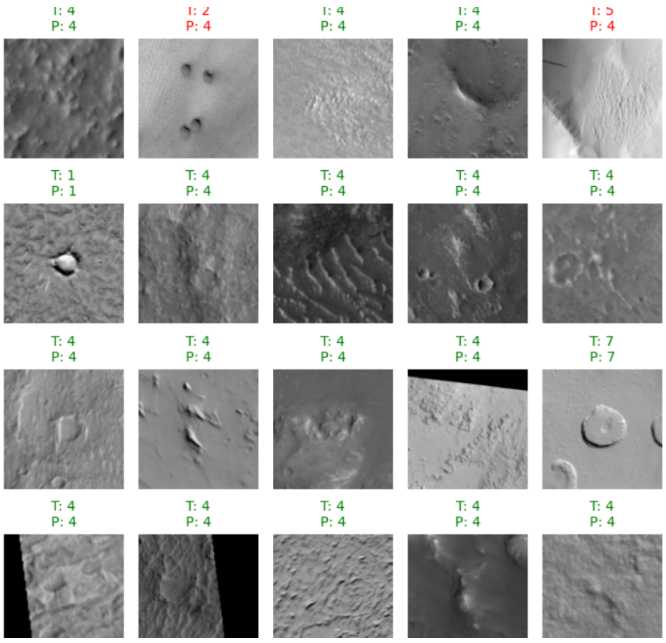

Fig. 6: True versus predicted labels for test images.

D. Per-Class Accuracy

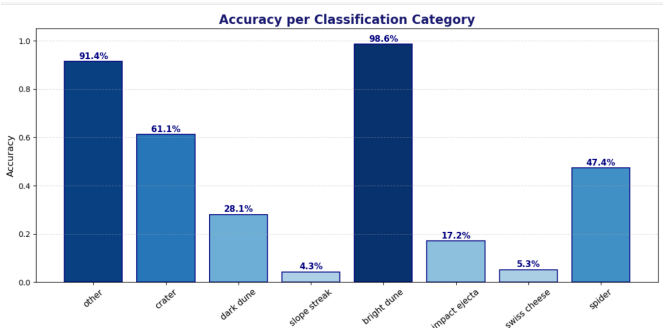

Fig. 7: Classification accuracy for each terrain category.

The per-class accuracy analysis examines how accurately the CNN model performs on individual classes of the eight classes of terrain. This will reveal how accurately the model works on certain classes of the eight classes of terrain.This has revealed significant variation in performance: Other (91.4%), Crater (61.1%), Dark Dune (28.1%), Slope Streak (4.3%), Bright Dune (98.6%), Impact Ejecta (17.2%), Swiss Cheese (5.3%), and Spider (47.4%). The results show that the model has performed exceptionally well on classes like Bright Dune and Other, but has difficulty in classes like Slope Streak, Swiss Cheese, and Dark Dune, which have less occurrence and subtle features.[8]

V. Discussion

The CNN model produced an excellent overall performance, which proves that deep learning techniques can identify Martian geological features successfully. There are minor differences in the model's performance, which are caused by the visual similarity of the classes and the data imbalance.[7]

Potential improvements include:

- Advanced data augmentation
- Transfer learning using deeper architectures (e.g., ResNet)
- Class rebalancing techniques
- Learning rate scheduling

.

VI. Conclusion

This project developed a convolutional neural network to automatically classify Martian surface features from over 73,000 HiRISE images. The model achieved a training accuracy of 90.79% and a test accuracy of 90.85%, with minimal loss difference (-0.0041), demonstrating strong generalization and stable learning.

Overall, the results have demonstrated the effectiveness of CNNs in feature extraction from planetary images and their potential in reducing manual inspection efforts. Future work could be directed to utilize transfer learning from deeper networks, sophisticated data augmentation, and class balancing methods.[9]

References

[1] NASA Jet Propulsion Laboratory, "HiRISE Mars Orbital Image Dataset," University of Arizona, Mars Reconnaissance Orbiter Mission.

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[3] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," Google Research, 2015. Available: https://www.tensorflow.org

[4] F. Chollet, "Keras," 2015. Available: https://keras.io

[5] Python Software Foundation, "Python Language Reference, Version 3.12," 2024. Available: https://www.python.org

[6] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020. (NumPy version used: 1.26)

[7] W. McKinney, "Data Structures for Statistical Computing in Python," Proceedings of the 9th Python in Science Conference, 2010. (Pandas version used: 2.2)

[8] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, 2007. (Matplotlib version used: 3.8)

[9] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2011. (scikit-learn version used: 1.4)